

JS

Basics, DOM modification, form validation

CREATED BY: ALEKSANDRA NOVÁK

What is JavaScript?

On **MDN JavaScript** we can read that JavaScript is **a programming language** that:

- is **high-level**
- is **dynamically typed** - JS has **data types,** however, they are associated with **values** not the **variable** itself. This means that variable's type does not have to be explicitly specified, and the variable may have assigned values of different types through its lifetime.

```
let value = "JavaScript" // string
value = 5 // number
```

- has **first-class functions** - functions may be treated as values, so they may be assigned to variable, passed as arguments and be return value. Read about **function declarations** and l**ook for code example in class materials.**

- has **type coercion** - the automatic conversion of value from one type to another when required by operation. Colloquially referred to as "JS magic". General rule:
  - **+**: if at least one value is string, it will concatenate
  - **-,*,/**: tries to convert to number
  - if impossible to convert, usually we get **NaN (Not a Number)**

```
"55" + 44 // "5544"
"55" - "44" // 11
55 - "44" // 11
"Java" - "Script" // NaN
"J" + "S" // "JS"
```

- traditionally is considered an **Interpreted language**, however, **modern JavaScript engines use Just-In-Time (JIT) compilation** to convert JS code into machine code during execution

    - **Compilation (Ahead-of-time):** source code → compiler → executable code (CPU architecture specific) → execution

    - **Interpretation (At runtime):** source code → interpreter→ execution (CPU executes the interpreter's machine code, e.g. written in cpp, which process the program's instructions)

    - **JIT (Just-In-Time):** source code → JIT compiler → machine code generated at runtime (CPU architecture-specific, however does not have other separate .exe) → execution

In other words: program starts → initial interpretation → JIT compiles at runtime selected code into machine code → CPU executes the generated machine code

JIT allows to reuse frequently executed code e.g. interpreter sees that some function (e.g calculate time difference) is called multiple times, so **JIT compiler** will compile it to machine code, which is stored and executed directly without a need for repeated interpretation.

Note that JS processes declarations before execution, so calling function id possible before its declaration (hoisting)

- is **garbage-collected** - automatically manages the memory. In opposite to C, does not require memory deallocation

- is **multi-paradigm** - supports ex. object-oriented, procedural, functional programming

- can be used in both browser and non-browser environments:
  - Non-browser (runs outside the browser):
    - Server-side (e.g.Node.js): HTTP requests handling, business logic, database connection
    - Other environments e.g. Adobe Acrobat
  - **Browser (client-side)**: **The one that will be used in this course**, **manipulates the DOM, adds dynamic behaviour to the page, adds interactivity**

HTML: WHAT IT IS

CSS: HOW IT LOOKS

JS: HOW IT BEHAVES

JS accesses HTML (and CSS) via DOM (and CSSOM)

Data types

JS has **7 primitive types**, **all immutable**, for our purpose the most important once are:

- **string** - represents text e.g. “JS”
- **number** - represents integers and floating point numbers, e.g. 55, 4.4
- **boolean** - true or false
- **null** - explicitly set to no value e.g, **let email = null;** **If we try to access element before DOM is ready we will get null!**
- **undefined** - no value, usually assigned automatically, e.g. just **let email;**

JS has **1 non-primitive type:**

- **Objects -** values in memory which can be referenced by variables. They are **mutable** by default and are only mutable values in JS. Here, we include:
  - **plain objects - { city: “Warsaw”}**
  - **arrays - [1, “a”, { city: “Warsaw”}]**
  - **Functions -** additional property of being callable
  - **Others:** e.g. new Date()

IMPORTANT:

- **Primitive types are copied by value**, changing new variable will not affect the old one.
- **Objects are copied by reference**. Both variables will point to the same memory address, changing value of one will change value of other

let vs var vs const

	var	let	const
scope	function/global	block ({})	block ({})
Reassignment	allowed	allowed	not allowed
Redeclaration in the same scope	allowed	not allowed	not allowed
Initialization	not needed	not needed	needed

The rule is to avoid var as it quite often lead to unexpected behaviour and error-prone code (e.g. hoisting).

Instead use:

- **const:** if the variable will not be reassigned. It does not make objects or arrays immutable as their content can still be modified

- **let**: when reassignment is expected

Remember that **mutability is not the same as reassignment! Mutable - can be modified after initial assignment (modyfying existing value)**

Operators

One of the most important operators are:

- **=** - assignment, e.g. `let name = "Kate"`
- **?** - conditional operator e.g. `const isOpen = hour > 13 ? "Closed" : "Open"`
- **?.** - optional chaining. Prevents error while trying to access null or undefined
- **??** - default value for null and undefined

Type vs type	==	===
Primitives	Loose equality. Comparison with type conversion	Strict equality. Comparison without type conversion
Objects	Compares objects identity	
Primitive vs Object	Loose equality. Can try convert object to primitive	Strict equality. Always false as types are different

```
let user = null;

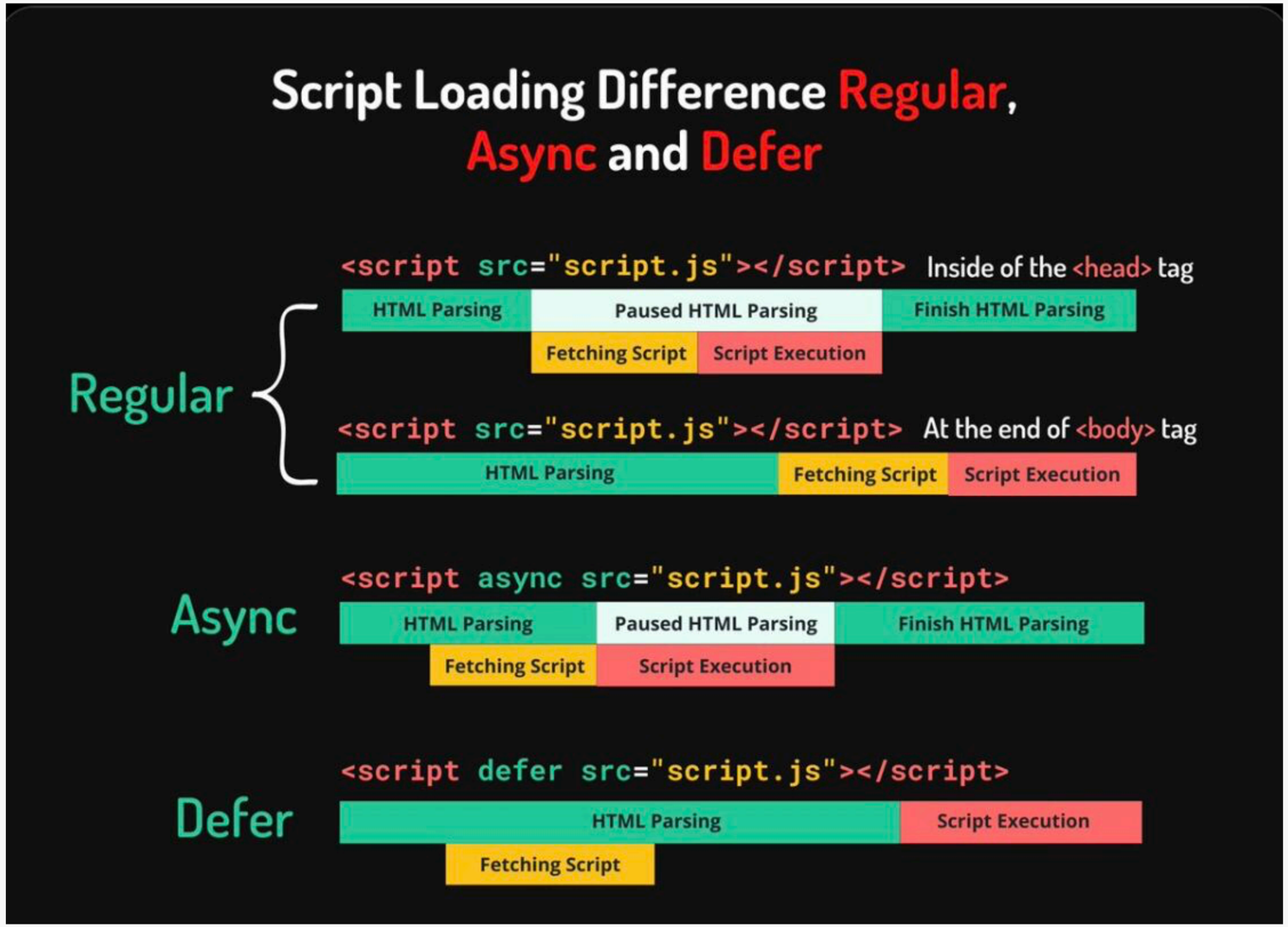
console.log(user); // null
console.log(user.name); //TypeError
console.log(user?.name); // undefined

console.log(user ?? "Kate") // "Kate
```

```
55 === "55" // false
0 === false // false
null === undefined //false
```

```
55 == "55" // true
0 == false // true
null == undefined //true
```

Linking Script



Source: https://www.linkedin.com/posts/razan-aboushi_javascript-js-defer-activity-7344615634111983616-Ybp5

There are 2 important events:

- **DOMContentLoaded event** - DOM is ready, some resources may still load. Usually enough for event listeners, elements manipulation
- **load event** - Resources finished loading. May be needed for e.g. some full layout animations

By using defer we do not need to wait for DOMContentLoaded event to be able to safely access DOM, as script is run after HTML parsing. It should be used however, with regular script linking in <head>

Events

Events are things that takes place in a system, and system can inform about them. The code can react accordingly to them.

Flow:
Event Trigger - action/condition that is a cause for event to occur (e.g. click) →
Event Fires - browser informs registered listeners about event →
Event Listener - listens for a specific event and uses the registered handler when event occurs →
Event Handler - function that executes a response to an event

Event Object

Object passed by the browser to an event handler, it has information about event and provides methods to control it:

- **event.type** - type of the event e.g click
- **event.target** - element which triggered the even, remains the same during propagation
- **event.currentTarget** - element that is currently executing handler
- **event.preventDefault()** - prevents browser's default action, e.g. form submitting
- **event.stopPropagation()** - stops event propagation

Handling Events

There are 3 ways to handle events:

addEventListener() `button.addEventListener("click",handleClick);`

- Allows more handlers for the same element for the same event type.
- Allows adding listener to capturing phase
- Separates JS from HTML
- can be removed by removeEventListener()

Event Handler Properties `button.onclick = handleClick;`

Separate JS from HTML but allows only on handler per event type for given element

Inline Event Handler `<button onclick="handleClick()">Click me!</button>`

Used usually only for some debugging purposes and in special cases

Event Propagation

It is a process that describe how an event traverse through the DOM tree. It has 3 phases:

1. **Capturing -** event goes down through the DOM to the target element. there maybe some handlers for capturing phase
2. **Target -** event is handled by target element’s listeners
3. **Bubbling -** event travels back up the DOM, triggering event handlers on ancestors elements if listeners were listening for the same event type

Remark: **this -** if event listener is a standard function this = event.currentTarget. rrow function do not have their own this.

Task 1

For given form (form.html, style.css) and JavaScript script for form validation, which will check input fields before allowing form to be submitted:

- Name cannot be empty or contain only whitespace. Should be between 3 to 30 characters
- Email cannot be empty and should be of a valid email format
- Password has to have at least 8 characters and include at least one uppercase letter, lowercase letter, number. Regex: /^(?=.*[a-z])(?=.*[A-Z])(?=.*[0-9]).{8,}$/

- Event handling:
  - use DOMContentLoaded event or <script> with defer attribute
  - use change event for fields validation
  - use submit event to validate form before submitting
  - display alert and ficus invalid field when validation fails
  - use .preventDefault() if any field is invalid

Can be used:

- document.getElementById()
- .value
- .trim()
- .focus
- alert()
- event.preventDefault()

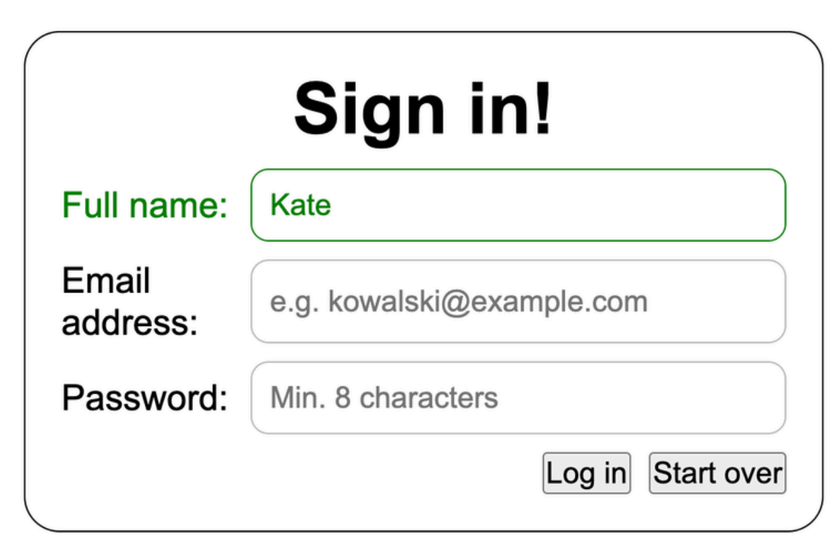


Task 2

Change the form validation from Task 1 by replacing the alert window with changing colors for border and label using CSS classes:

- Use keyup event for name input validation
- Use change event for rest of the fields
- if valid add valid class to the input and its label, if invalid and invalid class to the input and its label. Change color (green/red) for label text, inout border, and input text





Can be used:

- document.querySelector()
- .classList.add()
- .classList.remove()

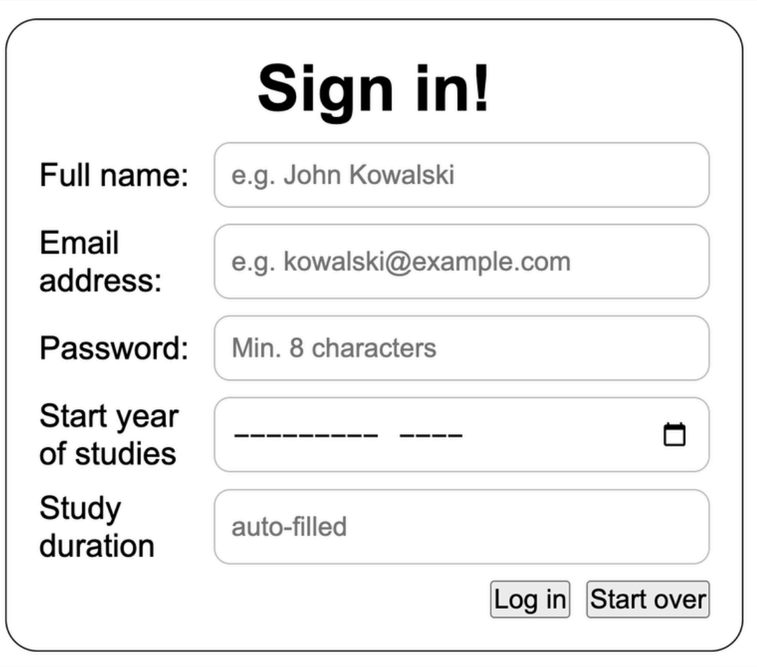
Task 3

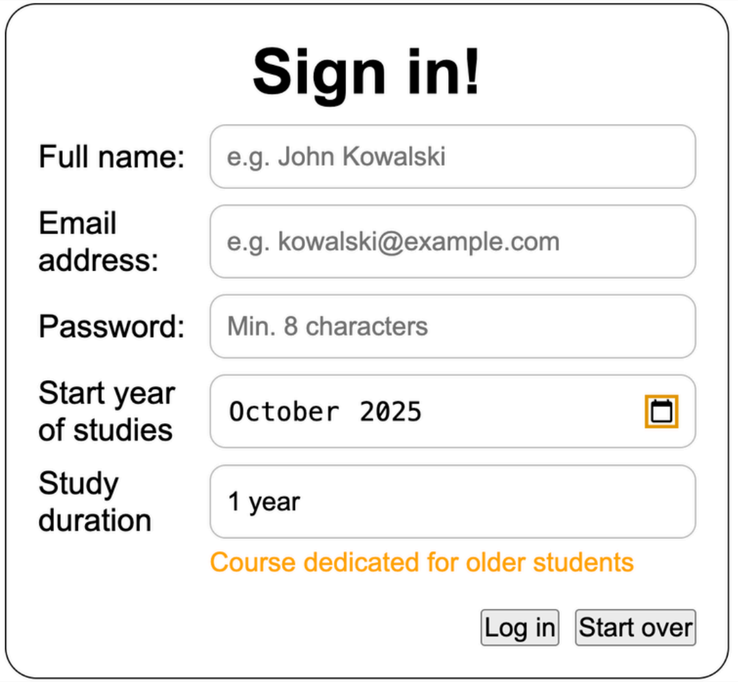
Extend Task2:

- adjust CSS
- add a month input for selecting start date of studies (use correct type)
- add a readonly input to display the calculated study duration (use correct type)
- use a change event to update duration when start date changes
- calculate duration by subtracting the selected year with current year. If duration is <= 1 display “1 year” in output field and dynamically create paragraph with message “Course dedicated for older students”. Otherwise display calculated number of years.
- Use reset event to remove dynamically created message

Can be used:

- new Date().getFullYear()
- document.createElement()
- .remove()
- .textContent

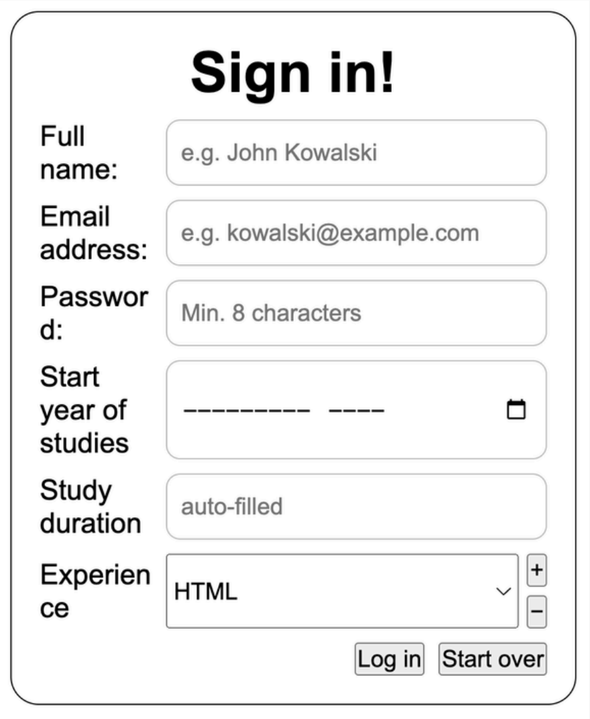


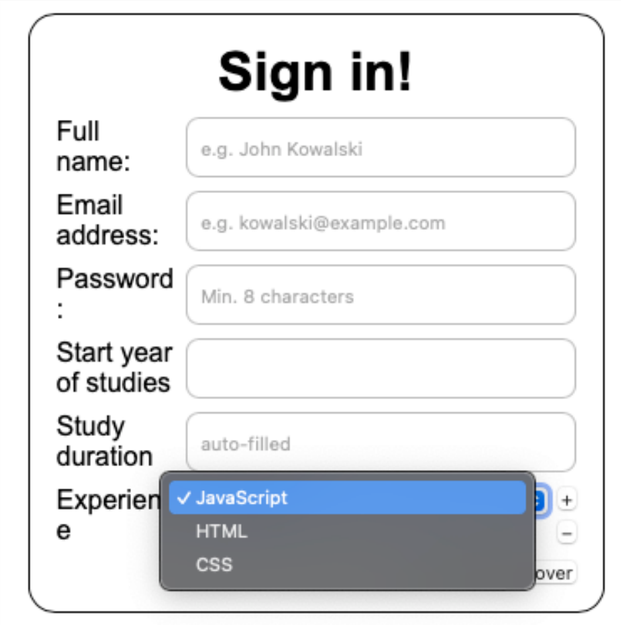


Task 4

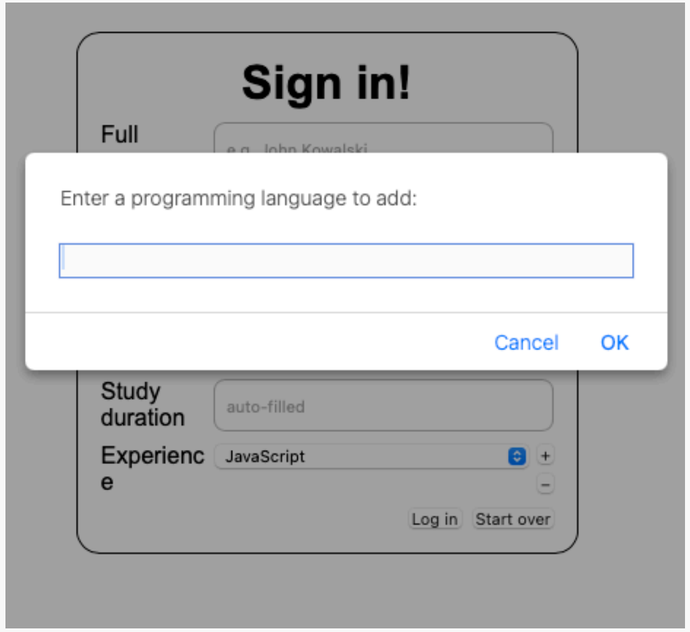
Extend the previous form by allowing user to add/remove dynamically programming language from the dropdown list:

- add s <select> element with initial list of programming languages
- add two buttons, make sure to adjust CSS
- Use click event to handle adding/removing
- Add(+) - use prompt() to get a language name, create <option> for dropdown and append it to dropdown
- Remove(-) - remove the currently selected option. Display alert when no option is selected
- Can be used: prompt(), createElement(), append child(), removeChild(), selectedIndex, select.options









Task 5

Extend the form so that it is possible to switch between sections:

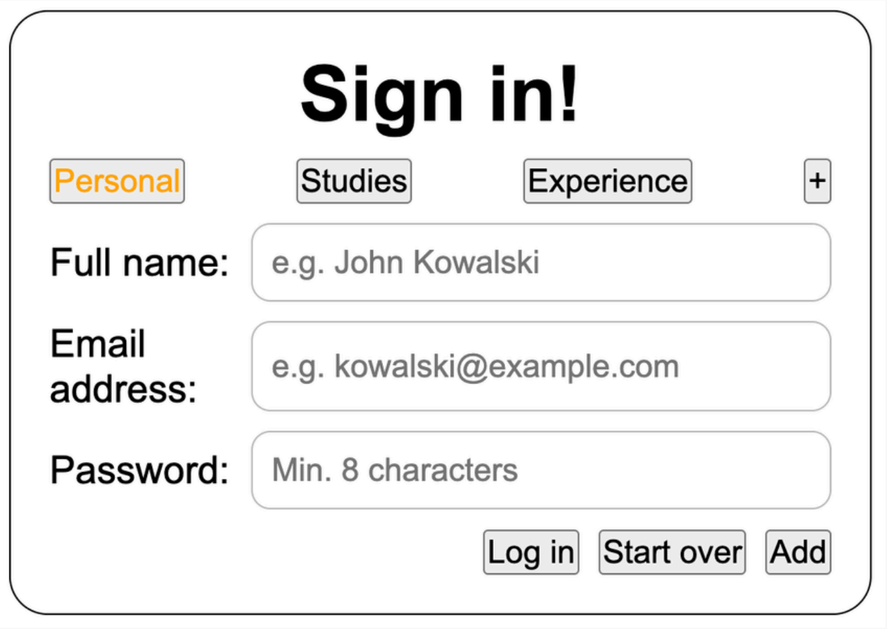
- Divide the form into 3 sections: Personal, Studies, Experience using <fieldset>
- Add navigation buttons (personal, studies, experience) with shared class and IDs
- Use click event to switch between sections
- Display only selected section and highlight the button

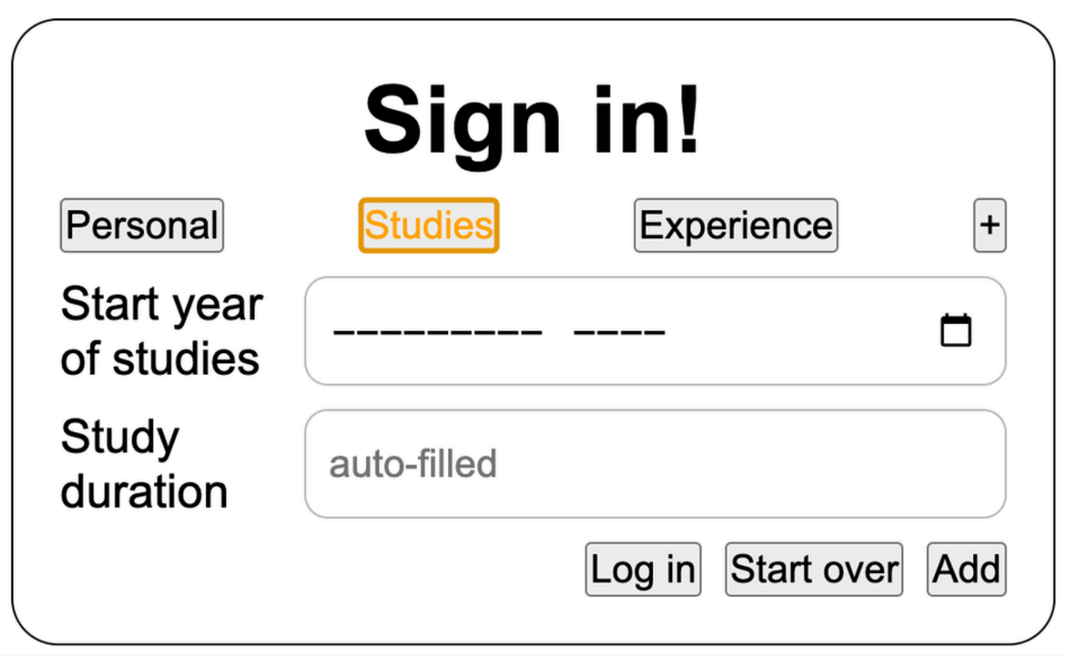
HINTS:

- some CSS class may be used for highlighting the button
- some CSS classes may be used to show selected section
- consider using naming pattern for button and section IDs. It will allow to do some “switching”. Clicked button’s ID can identify clicked section

Can be used:

- forEach()
- .classList.add()/remove()/.replace()





Disclaimer

- Tasks were adapted from the course materials “JavaScript - part I”, “JavaScript - part II”, tasks 1-6 and tasks 1-3, of the Web Application Programming/Programowanie Aplikacji WWW course by dr hab. inż. prof. uczelni Maciej Grzenda (PhD at the time of creation), website: http://www.mini.pw.edu.pl/~grzendam .

- The content presented in the presentation is consistent with the above-mentioned materials.

- The content is primarily based on the MDN documentation referenced on each slide, unless stated otherwise.

- All materials are provided for educational and non-commercial use only.